

Reviewer Pack — Minimal Rank of Graph Cocomplexes vs Resolution Proof Length...

Entry f7987e007c77 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 12:46:58 UTC

Minimal Rank of Graph Cocomplexes vs Resolution Proof Length for Tseitin Formulas

Entry ID: f7987e007c77 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 12:46:58 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Graph Cocomplexes) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity for Tseitin Formulas

Statement:

[‘For any graph G , the minimal rank of its cocomplex is poly-logarithmically lower-bounded by the resolution proof length of the Tseitin formula constructed from G , i.e., $\log(n)^k \leq R(G)$ where $R(G)$ is the resolution proof length for the Tseitin formula on G .’, ‘This implies that graphs with higher minimal rank will require exponentially longer resolution proofs.’]

Rationale (proposer’s reasoning):

[‘Graph cocomplexes provide a rich algebraic structure that captures topological information about graphs. This conjecture posits that this structure is closely related to the complexity of proving satisfiability using resolution, suggesting a potential way to characterize hard instances of Tseitin formulas.’, ‘If true, it could lead to new insights into the intractability of SAT and related problems.’]

Taxonomy category: TSEITIN_RES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f939ff25839608a3

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

If the average correlation coefficient between the minimal rank of the cocomplex and the resolution proof length for Tseitin formulas across at least 100 random graphs with n vertices is greater than or equal to 0.7, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "graph cocomplexes" AND "resolution proof complexity" AND Tseitin formulas" - "minimal rank" cocomplexes AND poly-logarithmically related TO resolution proof length" - "Tseitin formula construction" FROM graph AND impact ON minimal rank of cocomplex

Top relevant hits considered: - [http://arxiv.org/abs/1710.07343v1] Fractional Minimal Rank - [http://arxiv.org/abs/2208.01392v2] The Analytic Minimal Rank Sard Conjecture - [http://arxiv.org/abs/2406.15089v2] Sentences over Random Groups II: Sentences of Minimal Rank - [http://arxiv.org/abs/1408.6321v1] Crossing Minimization for 1-page and 2-page Drawings of Graphs with Bounded Treewidth - [http://arxiv.org/abs/1908.05124v2] Minimal Representations of Order Types by Geometric Graphs - [http://arxiv.org/abs/2503.13774v1] Jamming of Elastoviscoplastic fluids in Elastic Turbulence

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            for j in range(i+1, m):
                factor = A[j][i] / A[i][i]
                for k in range(n):
                    A[j][k] -= factor * A[i][k]
        return A

    def rank(A):
        A = gaussian_elimination(A)
        r = 0
        for row in A:
            if any(row):
                r += 1
```

```

    return r

def tseitin_formula(G):
    n = len(G)
    variables = {f'x{i}': i for i in range(n)}
    clauses = []
    for i in range(n):
        clauses.append([variables[f'x{i}']])
        for j in range(i+1, n):
            if G[i][j]:
                clauses.append([-variables[f'x{i}'], variables[f'x{j}']])
                clauses.append([-variables[f'x{j}'], variables[f'x{i}']])
    return clauses

def resolution(clauses):
    clauses = set(tuple(sorted(c)) for c in clauses)
    while True:
        new_clauses = []
        for c1, c2 in itertools.combinations(clauses, 2):
            if len(set(c1) & set(c2)) == 1:
                new_clause = list((set(c1) ^ set(c2)))
                if not any(all(l in clause for l in new_clause) for clause in clauses):
                    new_clauses.append(new_clause)
        if not new_clauses:
            break
        clauses.update(tuple(sorted(c)) for c in new_clauses)
    return len(clauses)

n = random.randint(5, 40)
G = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
A = []
for i in range(n):
    row = [0] * (n + 1)
    row[-1] = -1
    for j in range(n):
        if G[i][j]:
            row[j] = 1
    A.append(row)

R_G = resolution(tseitin_formula(G))
min_rank = rank(A)

return {
    "metric_name": "minimal_rank",
    "metric_value": min_rank,
    "instances_tested": 1,
    "conjecture_holds": min_rank >= math.log(n) ** 2,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:

```

```

    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot verify the conjecture with the pre-registered criterion. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17215
2	propose	ollama_remote	glm4:latest	0	0	12882
3	propose	ollama_remote	glm4:latest	0	0	13217
4	preregistration	ollama_remote	glm4:latest	0	0	9191
5	novelty	ollama_remote	glm4:latest	0	0	8210
6	novelty	ollama_remote	glm4:latest	0	0	10105
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13159
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12372
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10184
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10558
11	judge	ollama_remote	glm4:latest	0	0	11651

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 128745 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in research/audit/f7987e007c77.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/f7987e007c77.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/f7987e007c77.tar.gz (if generated)